aqua3S: Developing visualisation, early warning, and decision support components for a multi-partner WDN-focused digital twin service using FIWARE

G. Lewis[1*], B. Snider[2], L. Vamvakeridou-Lyroudia[3], A. S. Chen[4], S. Djordjević[5], D. A. Savić[6]

[1,2,3,4,5,6] Centre for Water Systems, University of Exeter. Harrison Building, North Park Road, Exeter, EX4 4QF, UK.

[3,6] KWR Water Research Institute. Groningenhaven 7, P.O. Box 1072, 3430 BB Nieuwegein, the Netherlands.

***Corresponding author: E-mail: g.lewis2@exeter.ac.uk**

ABSTRACT

The water industry is often and unfairly criticised for being relatively slow to adopt IT as part of its operating processes. To address this, the aqua3S project was tasked with developing open-source data definition, management, and processing, and applying them to the water sector. The aqua3S project sought to develop a data-centric approach using a FIWARE broker as a data hub for seven water distribution operator pilots across Europe and develop approaches for standardisation, security, and safety in the water distribution sector.

The purpose of this paper is to record the technical decisions, development activities and results generated for the work that was undertaken by University of Exeter in developing the visualisation, early warning, and decision support components of the project, demonstrate how these components integrate with the aqua3S theoretical goals of standardisation, security, and safety, and, moreover, demonstrate how the components integrate with the aqua3S FIWARE-centric broker architecture.

Keywords: FIWARE, digital twins, visualisation, decision support, early warnings

1 Purpose

The water industry is often and unfairly criticised for being relatively slow to adopt IT as part of its operating processes. To address this, the aqua3S project [1] was tasked with developing open-source data definition, management, and processing, and applying them to the water sector. The aqua3S project sought to develop a data-centric approach using a FIWARE [2] broker as a data hub for seven water distribution operator pilots across Europe and develop approaches for standardisation, security, and safety in the water distribution sector.

Standardisation was defined as the development of a single client-server application which would enable multiple pilots (water distribution regions) to collect, store and process sensor data using a FIWARE broker, with data being compartmentalised through FIWARE's *tenancy* model, restricting each pilot to *their* data.

Security was defined as ensuring that system data and functionality were only provided to registered users of the system. This was realised through the FIWARE Keyrock component [3], and enabled the definition of user accounts and passwords and user group functionalities, to limit functional access.

Safety was defined as the delivery of system functionality that would process sensor data and direct users to detect potentially hazardous events occurring through multiple visualisation presentation modes, the analysis of sensor data and providing functionality to support the undertaking of remediation activities where possible.

The purpose of this paper is to record the technical decisions, development activities and results generated for the work that was undertaken by University of Exeter in developing the visualisation, early warning, and decision support components of the project.

2 Materials and Methods

The visualisation, early warning, and decision support components were developed for this project in Python using Flask to implement the components as web services as part of a larger system comprising of intercommunicating docker components. In terms of model, view and controller (MVC) architecture, FIWARE acted as the model repository for the system, the visualisation component acted as the view, leaving the controller component responsible for processing early warning and decision support functionalities. This was achieved by collecting sensor data from the broker, process it, and store results back on the broker, either as updates to the sensor smart model or using other smart models. The visualisation service would retrieve processed sensor data from the broker and display results in a browser.

2.1 FIWARE broker

A FIWARE broker forms the core of the aqua3S architecture, giving the overall architecture of a data hub. FIWARE brokers are implementations of CMI-based standards [4] for ngsi-ld/v1 or ngsi-ld/v2, describing a CRUD-based data lifecycle of create, retrieve, update, and delete for smart models, realised as JSON-LD key-value pairs. FIWARE maintains a repository of smart models [5], that developers can use, extend or re-implement for their projects, The broker interface is realised through HTTP requests using post-based requests for smart model creation, get for retrieval, patch to update, and delete for deletion.

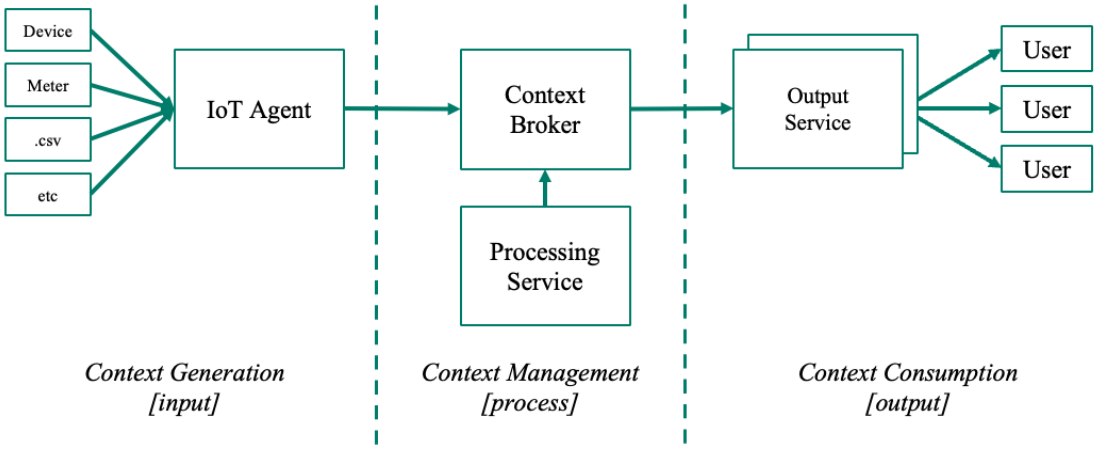

Figure 1 Prototypical FIWARE architecture

Figure 1 outlines a prototypical FIWARE implementation with three core functions: context generation, context management and context consumption. For digital twins, real-world data is collected as inputs to the IoT Agent in the context generation phase. The goal of this phase is to collect data and create / generate smart model instance (contexts) representations of the data to be stored in the broker. For the aqua3S project data sources are pilot-specific, with some pilots providing data through periodic historic csv spreadsheet files, whilst others provide data through live sensor readings.

The context management function of the broker describes functionality where contexts can be processed to create new data and/or contexts. Typically, this describes activities where 'dumb' data is collected from the IoT Agent and then processed, potentially with other data, to create insights, e.g., sensor data may be collected by the IoT Agent and is processed later to determine is the values received are in or out of range.

Finally, the context consumption function describes activities where broker data is processed as output. This is likely to be situation specific and may include activities like producing daily reports, providing data for webapps and so on.

2.2 Early warning component

The early warning component was responsible for processing sensor data and localising resultant issues. The system generated three types of early warnings: alerts, anomalies and EPANET anomalies. Alerts were generated when sensor readings fell outside of user-provided under-bottoming or over-topping limits.

Simple anomalies were implemented through the analysis of bucketed data. Each sensor had a set of historic min/max values constructed from multiple weeks of data and compartmentalised into a typical 7-day / 8-hour set of buckets. Sensor readings that fell outside of the min/max range for a given time bucket would raise an anomaly, Figure 2 details typical anomaly settings.

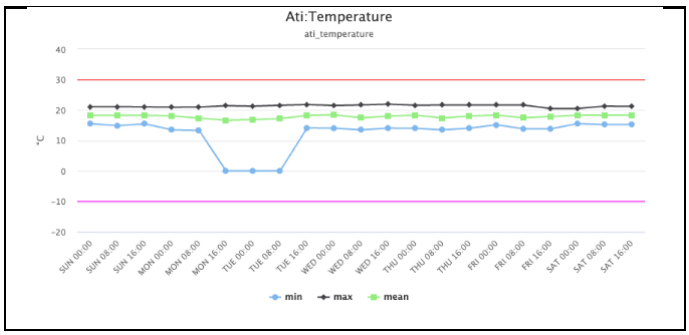

Figure 2 anomaly settings

The advantage of anomalies over alerts is that anomalies give more sensitivity (based on historic performance) than alerts which can be regarded as something of a blunt instrument. From the anomaly settings in Figure 3, the black and blue traces represent maximum and minimum anomaly limits whilst the red and purple lines represent alert maximum and minimum limits. The space between black and red and blue and purple line pairs represents the extra sensitivity that anomaly detection enables.

EPANET anomalies used a similar approach to *simple* anomaly detection model, with two key exceptions. Firstly, historic sensor data was simulated using an EPANET model with noise applied to sensor results to create a rich backlog of data with small scale variance. Secondly, anomaly detection was implemented through an exponential weighted moving average algorithm.

Positive EPANET anomaly results were used as an input to the anomaly localisation algorithm, as the EPANET anomaly result would inform users where the anomaly / leak was detected, but not where it had occurred. To localise leak detection, the EPANET model was initially used to generate large training sets of normal and leaking network data. This was necessary as the pilot WDNs tended to be far too reliable and leak-free to use as meaning training data. Once trained, the classification model was used to localise likely leak regions with leak probabilities. This data could then be passed to leak crews to localise their leak detection activities, Figure 3, with the red circle showing the likely leak location.

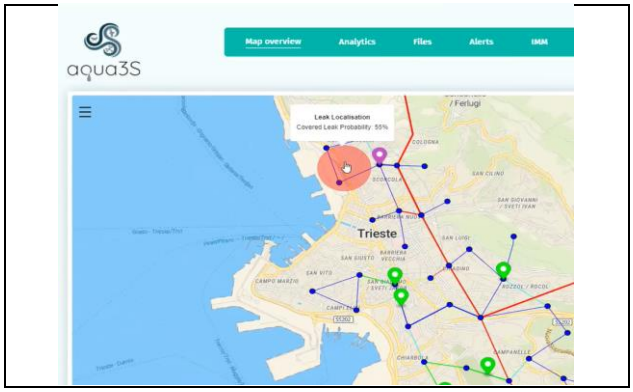

Figure 3 aqua3S anomaly localisation

Early warning processing was implemented as part of the *context management* stage, in that the alert and anomaly processes would collect data from the broker and process it to create new broker data as alert and anomaly status smart model instances.

2.3 Decision support component

The decision support component was intended to be used once leak crews had manged to locate the leak in the physical WDN as part of the early warning component activities, as described in section 2.2, and was responsible for evaluating and presenting courses of action for addressing leakage and provided users with a set of possible actions based on multiple evaluation criteria: pressure loss in the network, unmet demand, discoloration, and overall leak volume, Figure 4.

The decision support component was implemented as a genetic algorithm which developed multiple interventions strategies and evaluated them through EPANET simulations, using the evaluation criteria above as a fitness function.

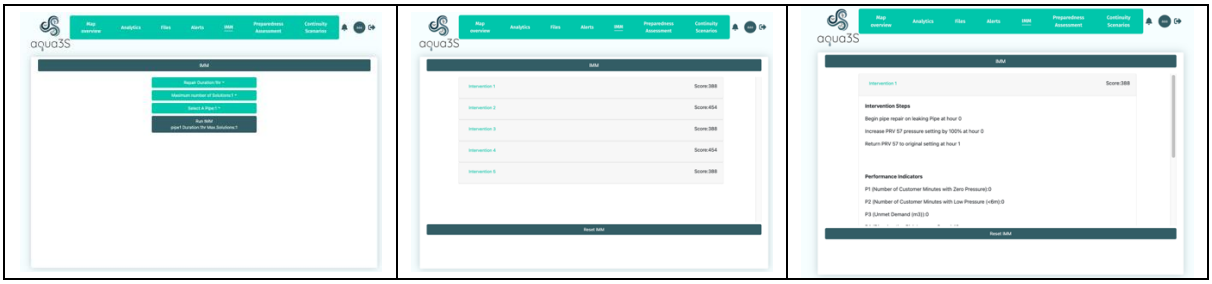

Figure 4 aqua3S decision support, set-up (left), overall results (centre), result detail (right)

2.4 Visualisation component

The visualisation component was implemented as a *context consumption* component and designed to provide users with early-warning and decision support information through multiple styles of visualisation. The component was implemented as a set of web pages that were a subset of the overall aqua3S webapp and was realised as a Flask webserver.

Early-warning provided three modes of visualisation; geographic using maplibre [6] to display pilot sensors as colour-coded markers, charting using Highcharts [7] to display historic sensor readings over multiple timeframes and textual to display sensor readings in a tabular form, Figure 5, whilst decision support implemented the textual format described in Figure 4.

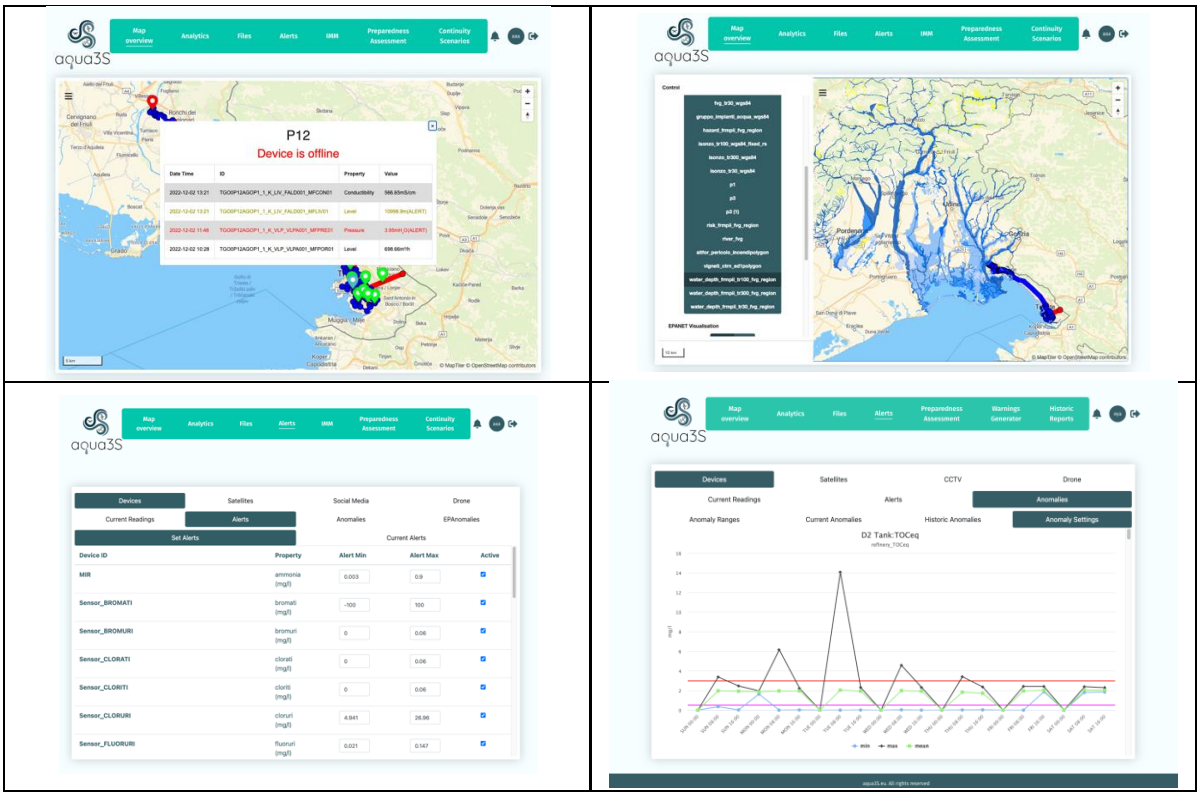

Figure 5 aqua3S modes of visualisation, geographic display with sensor detail (top left), flood risk overlay (top right), textual alert display (bottom left) and anomaly chart (bottom right).

3 Results and Discussion

3.1 Overall results

The iterative nature of delivering periodic user prototypes enabled the application to be developed in a manner that was generally well-aligned with user needs, both explicit and those discovered during development, and gave the research project a clear user focus. Towards the end of the project, it became apparent that aqua3S could serve as a scenario planning and training tool for operators.

3.2 FIWARE

Conceptually, FIWARE makes a huge amount of sense, particularly with its simplified interface and, for researchers, the removal of SQL-like functionality for a clear CRUD interface that is geared around entities and temporal data does much to stop water engineering projects from becoming database projects.

One huge challenge with the project in terms of FIWARE was dealing with the issues of the delay in getting a broker working reliably, something which took the best part of 12 months and made it very difficult to develop functionality that relied on broker-based data. To a degree this was mitigated through the development of mock broker-like interfaces to simulate context provision for *context processing* and *context consumption* activities, but it remained an issue until a development broker was developed.

3.3 FIWARE performance

During synthetic data generation activities, it was discovered that FIWARE's performance was limited, to a degree, by the number of HTTP requests it could process in each timeframe. For normal use of FIWARE that wasn't an issue, but for occasions where a broker was being loaded with a lot of historic data through multiple patch requests, e.g., 12-months of sensor readings at 15-minute intervals would generate around 36,000 requests, which tended to never complete. This was addressed in section 3.4, through the development of a specialised broker request that could accept arrays of patch data.

This was also an issue, albeit to a lesser extent, with our initial smart model architecture. We followed a SOLID 'single responsibility' approach for the development of smart models which resulted in separate models for device reading data, alert, anomaly and EPANET anomaly settings and status. The plan being that early warning alert and anomaly models would split data between settings and status, so that status data could be stored temporally to view status change over time.
This created two problems: 1) device data took much longer to load as each device effectively became 5 separate models, running into the HTTP request 'bottleneck', and 2) the anomaly and alert processing functions could not be made aware of the historic state of previous processing, so each time data was processed all the device, state and setting data needed to be retrieved, taking several minutes to update and leading to a lot of situations with drift between overserved sensor data and state data.
This was ultimately replaced with a simpler approach that stored all the senor-related data in the sensor without temporal state and setting data, resulting in a far simpler operation and timely data presentation.

3.4 Development context broker

To address the issues raised in section 3.1, a lightweight development context broker was developed. It was realised as a Flask server with an SQLite database backend and implemented the nsgi-ld/v1 interface that was being used on the project. This enabled our development to continue whilst the actual FIWARE broker was integrated and gave a great deal of insight into the operation of FIWARE services.

3.5 Synthetic data generation

We ran into two key issues with data on the project: firstly, pilot data was not available until the FIWARE broker implementation was established and working reliably and, secondly, pilot data was generally 'in range', with edge case data (leaks, over-topping, under-bottoming) events being incredibly rare.

These issues were addressed with the development of synthetic data generation which would allow us to create plausible data to 1) test FIWARE data pipelines, 2) test algorithms around *bulk* or operational data loads, and 3) respond to edge cases.

The approaches taken to data generation were largely dependent on the use-case(s) that were being addressed, EPANET proved incredibly useful for this service and delivered training and testing data for EPANET-based anomaly detection and localisation algorithms. However, it was generally limited to pressure and flow data. Moreover, as an approach it was limited to the pilots that were able to provide calibrated EPANET models as it appears to be a common practice to consider EPANET purely as a WDN topology and ignore its hydraulic capabilities and requirements.

For more general synthetic data generation, use was mode of the historic sensor data that drove the anomaly detection which facilitated both in range and anomalous values to be generated.

3.6 3-step development approach

From a development perspective, the project highlighted the benefit of using a 3-step approach, with local development, remote testing, and operations as three distinct development-operational environments.

Undertaking much of the development in isolated local development environments allowed developers to create and destroy broker data without any impact on the overall system, certainly not the live system, and allowed development to occur without having the fallout of other developer's development activities. However, this approach did raise some issues with the broker, particularly in terms with attempts to bulk-load historic data.

The remote testing environment allowed developers to perform longer-term testing on components, particularly where data needed to be collected over multiple days, as local computers tended to be shut down every evening and proved to be a very productive environment for detecting issues that only occurred 'over time'.

4 Conclusion and Perspectives

The project has successfully completed, largely from a combination of clear goals and communications, a motivated team across the project and pilots, and a highly committed project manager. Like all development projects, the project was not plain sailing and ran into a lot of small-scale road bumps that were, fortunately, addressed.

4.1 Synthetic data-centric development environment

The use of synthetic testing data and a development broker allowed us to undertake meaningful development before a working and reliable broker data pipeline was in place. Whilst our re-engineering of context broker functionality can be viewed as a questionable activity, and we agree with that, there wasn't an alternative at that point.

Being able to create data for testing algorithms, in the broadest senses of being able to verify data flow, work with bulk and normal data, and create edge-case / out-of-range data, enabled working algorithms to be developed, and was a key success factor for our work on the project.

4.2 Test-driven development

One of our biggest regrets on the project was introducing formal unit testing very late during development and it is felt that this would have significantly improved our development work by automating much of our regression testing. Initial development was undertaken by building FIWARE components and running them as separate applications, though this tended to be difficult to orchestrate debugging activities.

As a halfway-house, much of the application functionality was moved into a single Python package with an integrated testbed. This enabled the manual control dataflow (from generation, to processing and consumption) and debug it in a single application instance.

4.3 User centricity and KISS

From our experiences with aqua3S, a lot of our issues came from inadvertently trying to over-complicate the development process and/or the functionality that users needed or wanted. The most obvious case of this was using SOLID to drive smart model development and it ultimately became apparent that our users did not care for the over-complicated (and slow) approach that we had taken to develop the early warning components. Replacing this with a far less complex version was a simple undertaking and addressed a lot of pilot concerns.

Whilst this is an obvious example of ignoring users (as the pilot users), it is worth remembering that developers are also users and there is a lot of value in considering their needs, in terms of having stable environments to quickly and iteratively develop and test functionality. We can see that approaches like the 3-step development model, synthetic data generation, and testbed development enabled developers to make the best use of their limited time on the project to meet both their and the project's objectives.

5 Acknowledgements

The work presented in this paper was funded by EC H2020 aqua3S(GA832876) project.

6 References

[1] "aqua3S Project Homepage," [Online]. Available: https://aqua3s.eu/. [Accessed 9 march 2023].

[2] "FIWARE Homepage," [Online]. Available: https://www.fiware.org/. [Accessed 9 March 2023].

[3] "Keyrock Identity Manager," [Online]. Available: https://keyrock-fiware.github.io/. [Accessed 9 March 2023].

[4] "ETSI Context Information Management (CIM); NGSI-LD API," [Online]. Available: https://www.etsi.org/deliver/etsi_gs/CIM/001_099/013/01.01.01_60/gs_CIM013v010101p.pdf. [Accessed 9 March 2023].

[5] "FIWARE Smart Data Models," [Online]. Available: https://smart-data-models.github.io/data-models/. [Accessed 15 March 2023].

[6] "MapLibre Open Maps SDKs," [Online]. Available: https://maplibre.org/. [Accessed 9 March 2023].

[7] "Highcharts Charting Library," [Online]. Available: https://www.highcharts.com/. [Accessed 9 March 2023].

[8] "EPANET Homepage," [Online]. Available: https://www.epa.gov/water-research/epanet. [Accessed 9 March 2023].

[9] "Fiware4Water Project Homepage," [Online]. Available: https://www.fiware4water.eu/. [Accessed 9 March 2023].